

Linear Models 6: Residual Analysis and Transformations Lab

Dr. Matteo Tanadini

Applied Machine Learning and Predictive Modelling 1, HS25 (HSLU)

Contents

1	The “farms revenue” dataset	2
2	The “Ozone” dataset	7
3	Intrepration of coefficients in models with transformation	15
3.1	log(y) case	15
4	Appendix	17
4.1	log(y) are “multiplicative models”	17
4.2	“Nature” is very often multiplicative (especially for amounts)	17
4.3	log(y) ~ log(x) case	17
4.4	Other advantages of log(y) models	18
4.5	Why not taking log of e.g. sbtp? (*)	19
5	Session Information	20

This Lab IS NOT to be considered exam material!

This Lab is provided for the sake of completeness. The exam will NOT contain any questions about this Lab.

1 The “farms revenue” dataset

```
# d.farms <- read.table("http://stat.ethz.ch/Teaching/Datasets/WBL/farmsAus.dat",
#                       header = TRUE)
## or
d.farms <- readRDS("Farms_Australia.RDS")
str(d.farms)
```

```
'data.frame':  451 obs. of  4 variables:
 $ region      : int  111 111 111 111 111 111 111 111 111 111 ...
 $ industry     : Factor w/ 5 levels "cattle","mixed",...: 4 3 2 5 2 3 2 4 4 4 ...
 $ expenditure: num  115.1 75.4 378.9 433.6 347.4 ...
 $ revenue      : num  147.7 82.9 442.7 649.6 407.8 ...
```

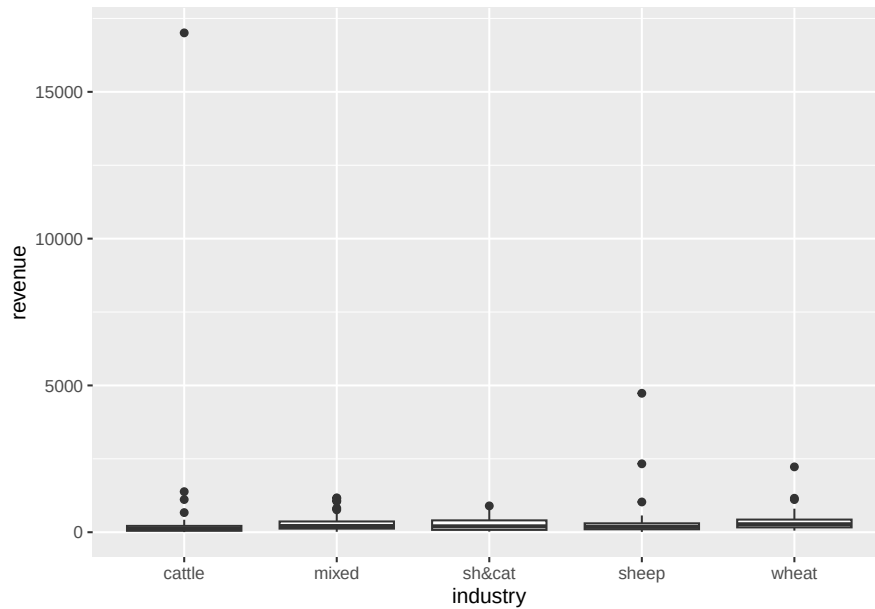
```
head(d.farms)
```

	region	industry	expenditure	revenue
1	111	sheep	115	148
2	111	sh&cat	75	83
3	111	mixed	379	443
4	111	wheat	434	650
5	111	mixed	347	408
6	111	sh&cat	328	473

The data set contains information about 451 farms in Australia. For each of them, the annual revenue, the annual expenditure, the region and the industry was recorded.

We focus here on the relationship between the response variable *revenue* and the *industry* in which each farm is active. Let's visualise the data.

```
library(ggplot2)
ggplot(data = d.farms,
       mapping = aes(y = revenue,
                     x = industry)) +
  geom_boxplot()
```

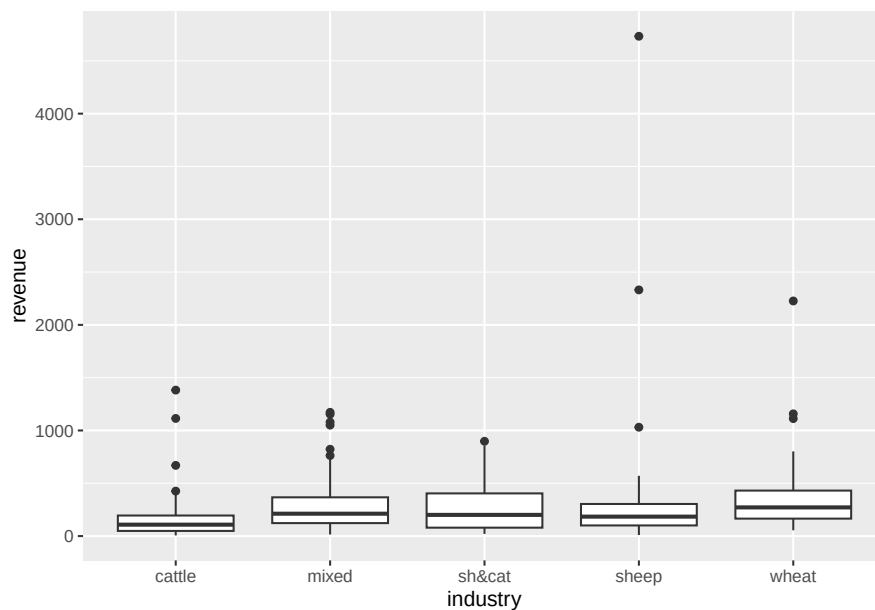


There is a very clear outlier in this data. Indeed, there is one farm in the *cattle* industry that has a revenue, which is way larger than any other farm.

A naive way to proceed would be to remove this observation and simply proceed with the analysis.

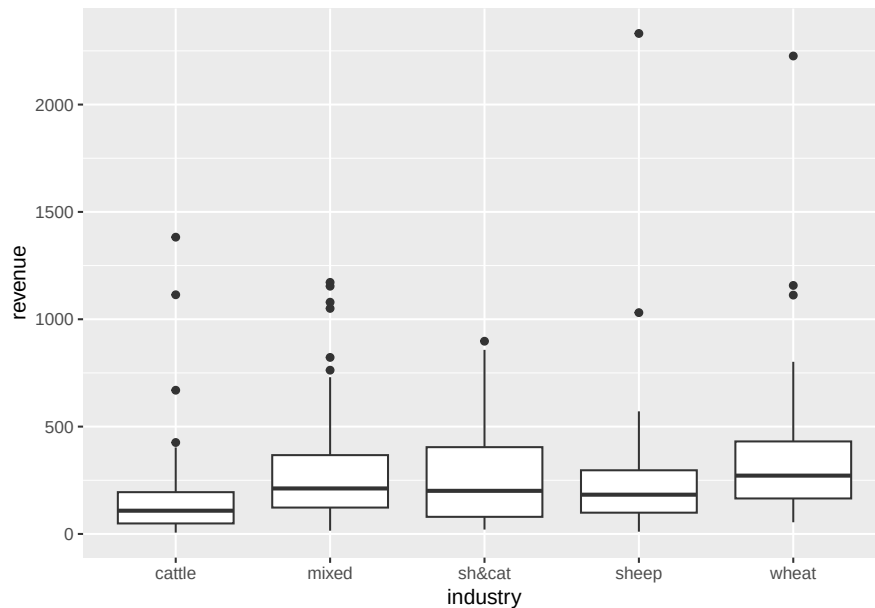
Let's remove this "outlier" and reproduce the graph above before fitting a model.

```
library(dplyr) ## contains the filter() function
ggplot(data = filter(d.farms, revenue < 10^4),
       mapping = aes(y = revenue,
                     x = industry)) +
  geom_boxplot()
```



Actually, once we remove the outlying observation...the next one pops up! Let's try to remove the observation in *sheep* group that appears to be an outlier.

```
ggplot(data = filter(d.farms, revenue < 4000),  
       mapping = aes(y = revenue,  
                     x = industry)) +  
  geom_boxplot()
```

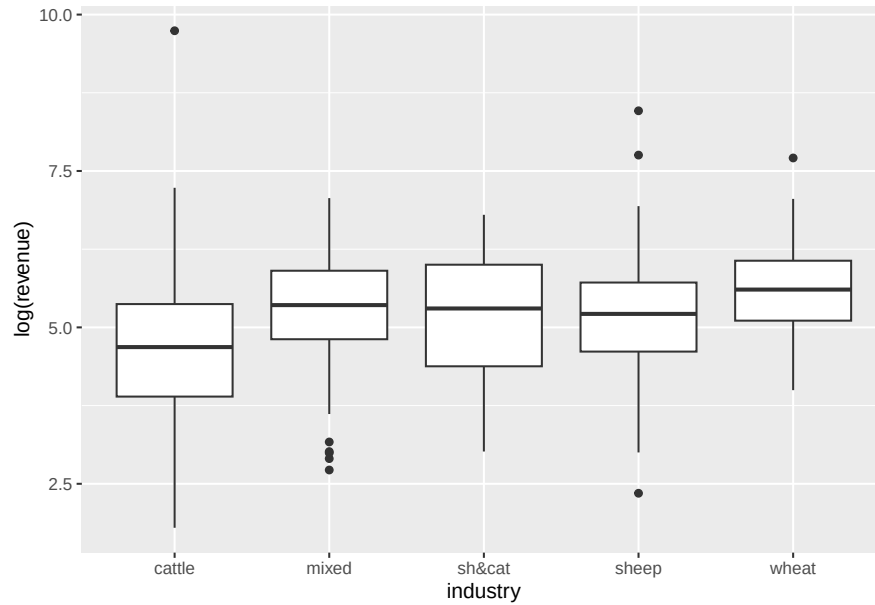


Again, other observations are now “outliers”. Indeed, the real solution here is to log-transform the response variable. As a matter of fact, the response variable is an “amount” and should therefore be transformed with the $\log(y)$ -transformation.

Another hint that $\log(y)$ -transformation is needed, is the skewness of the data. Note that the “outliers” (i.e. the black dots) are only present on the upper part of the boxplots plots, but not in the lower part.

Let's try to log-transform and then reproduce the plot again.

```
ggplot(data = d.farms,  
       mapping = aes(y = log(revenue),  
                     x = industry)) +  
  geom_boxplot()
```

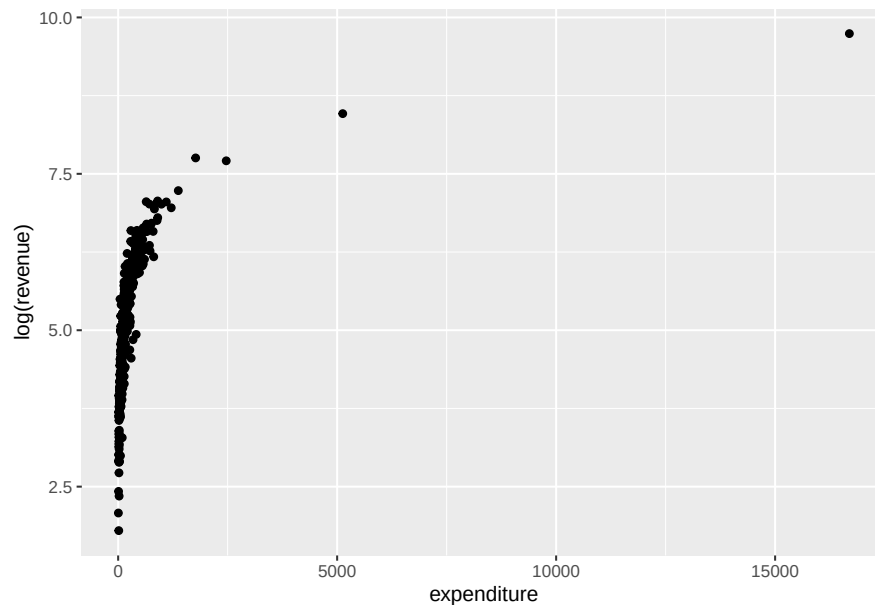


The transformation of the response variable worked well. Indeed, the outliers are not present and the data look symmetric. Note that the $\log()$ function implements the natural logarithm.

The dataset contains two other predictors: *region* and *expenditure*. A natural question that may raise here is whether *expenditure* that is also an amount needs to be transformed? More in general, we may ask ourselves whether predictors that are amounts should be log-transformed too?

Let's first visualise the relationship of $\log(\text{revenue})$ against *expenditure* with a scatterplot.

```
ggplot(data = d.farms,
       mapping = aes(y = log(revenue),
                     x = expenditure)) +
  geom_point()
```



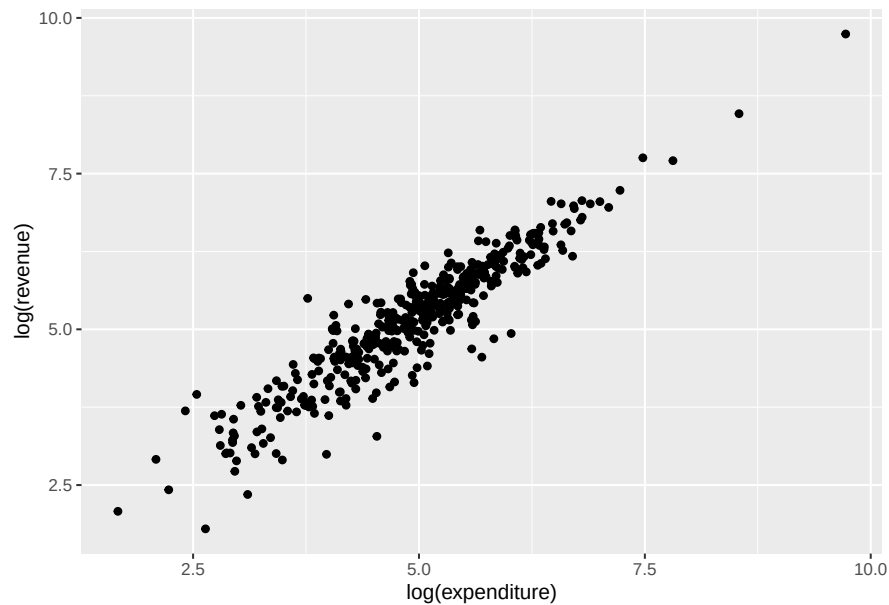
Question: What patterns do you observed on this graph? Would you transform *expenditure* as well?

There are two things to say about the previous plot:

- the relationship is not linear
- *expenditure* has also a skewed distribution

Let's try to log-transform *expenditure* as well.

```
ggplot(data = d.farms,  
       mapping = aes(y = log(revenue),  
                     x = log(expenditure))) +  
geom_point()
```



Voilà the relationship between two variables is nice and linear in the log-log scale. In addition, there are no more observations that would be very influential here (compare to the previous graph).

So, in general, **you often may want to transform both: response variables and predictors that are amounts**. In very many situations we must imperatively transform response variables that are amounts. If we don't do so, then the model assumptions are not fulfilled.

Note that the interpretation of the model coefficients when response and explanatory variables are transformed is addressed further down in this document.

2 The “Ozone” dataset

Let’s get the data. The data is located in the package *{gss}* that you may need to install.

```
# install.packages("gss")
library(gss)
data(ozone)
## OR
# ozone <- readRDS("Ozone.RDS")
```

Let’s have a look at the help file for this dataset.

```
?ozone
```

Let’s check the data structure.

```
str(ozone)

'data.frame':  330 obs. of  10 variables:
 $ upo3: num  3 5 5 6 4 4 6 7 4 6 ...
 $ vdht: num  5710 5700 5760 5720 5790 5790 5700 5700 5770 5720 ...
 $ wdsp: num  4 3 3 4 6 3 3 3 8 3 ...
 $ hmdt: num  28 37 51 69 19 25 73 59 27 44 ...
 $ sbtp: num  40 45 54 35 45 55 41 44 54 51 ...
 $ ibht: num  2693 590 1450 1568 2631 ...
 $ dgpg: num  -25 -24 25 15 -33 -28 23 -2 -19 9 ...
 $ ibtp: num  87 128 139 121 123 182 114 91 92 173 ...
 $ vsty: num  250 100 60 60 100 250 120 120 120 150 ...
 $ day : num  3 4 5 6 7 8 9 10 11 12 ...
```

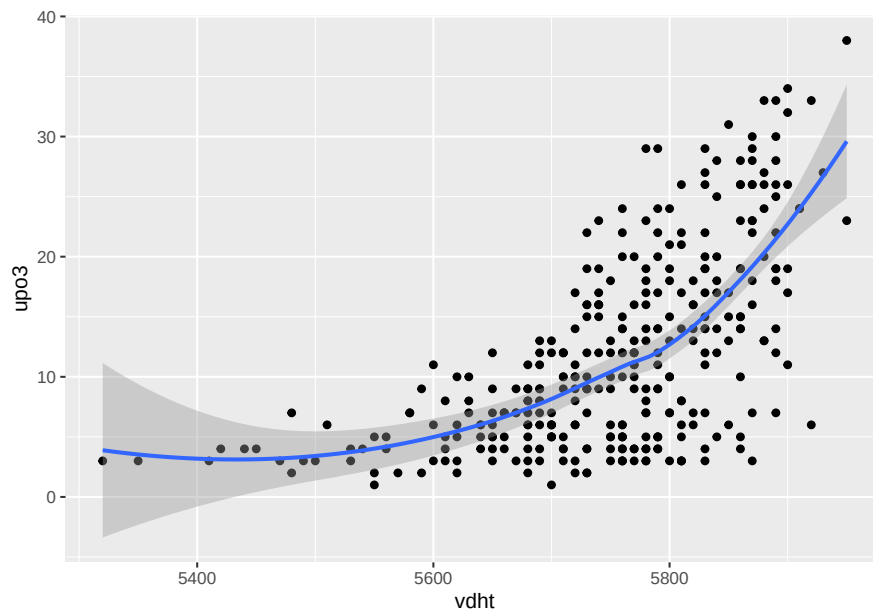
```
head(ozone)
```

	upo3	vdht	wdsp	hmdt	sbtp	ibht	dgpg	ibtp	vsty	day
1	3	5710	4	28	40	2693	-25	87	250	3
2	5	5700	3	37	45	590	-24	128	100	4
3	5	5760	3	51	54	1450	25	139	60	5
4	6	5720	4	69	35	1568	15	121	60	6
5	4	5790	6	19	45	2631	-33	123	100	7
6	4	5790	3	25	55	554	-28	182	250	8

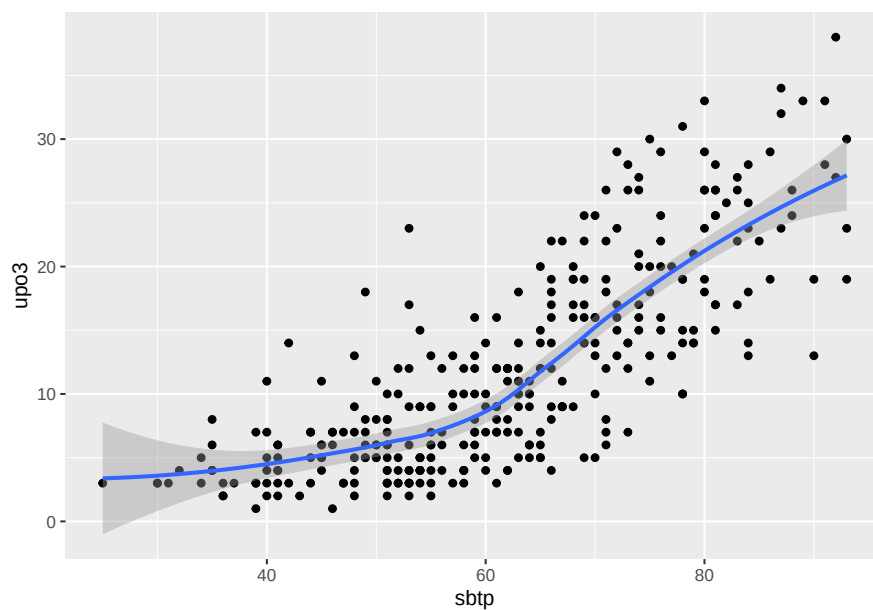
The response variable we want to model is ozone concentration (i.e. *upo3*). For didactic reasons, we are going to use only three predictors: *vdht* (air pressure), *sbtp* (air temperature), *ibht* (inversion height).

Let’s start with the exploratory graphical analysis.

```
# vdht
ggplot(data = ozone,
       mapping = aes(y = upo3,
                     x = vdht)) +
  geom_point() +
  geom_smooth()
```



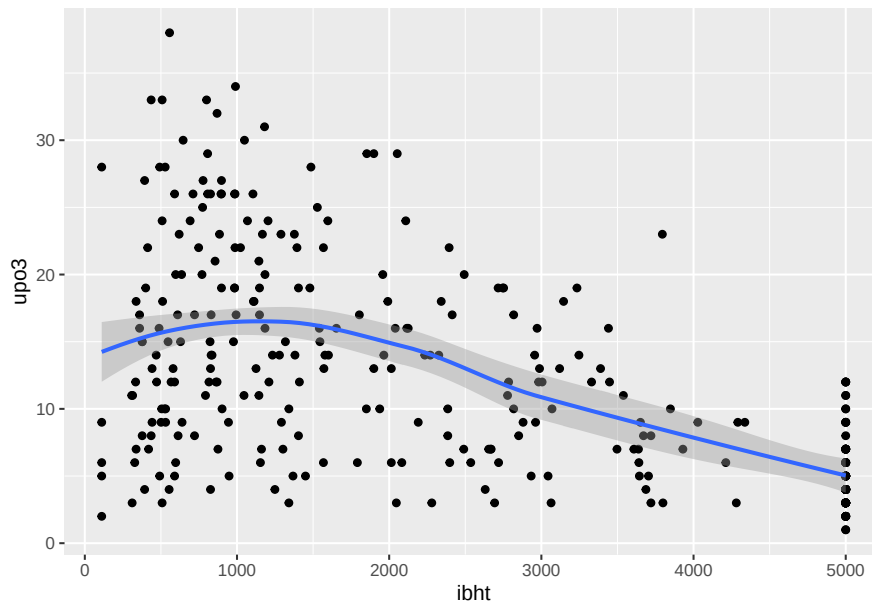
```
#
# sbtp
ggplot(data = ozone,
       mapping = aes(y = upo3,
                     x = sbtp)) +
  geom_point() +
  geom_smooth()
```



```
##
# ibth
ggplot(data = ozone,
       mapping = aes(y = upo3,
                     x = ibht)) +
```



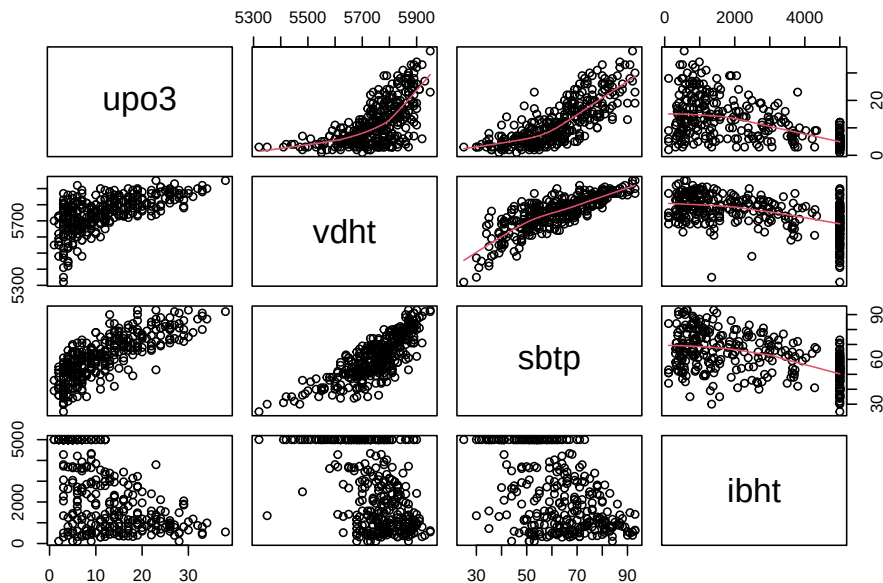
```
geom_point() +  
geom_smooth()
```



The relationships between the response variable and the predictors do not seem to be linear. In addition, it looks like the variability of the errors may not be constant. The problem of heteroscedasticity is particularly evident for the *vdht* predictor.

Note that in case where very few predictors are available we can also use the *pairs()* function to visualise the data.

```
## all plots together (quick and dirty)  
pairs(upo3 ~ vdht + sbtp + ibht, data = ozone, upper.panel = panel.smooth)
```



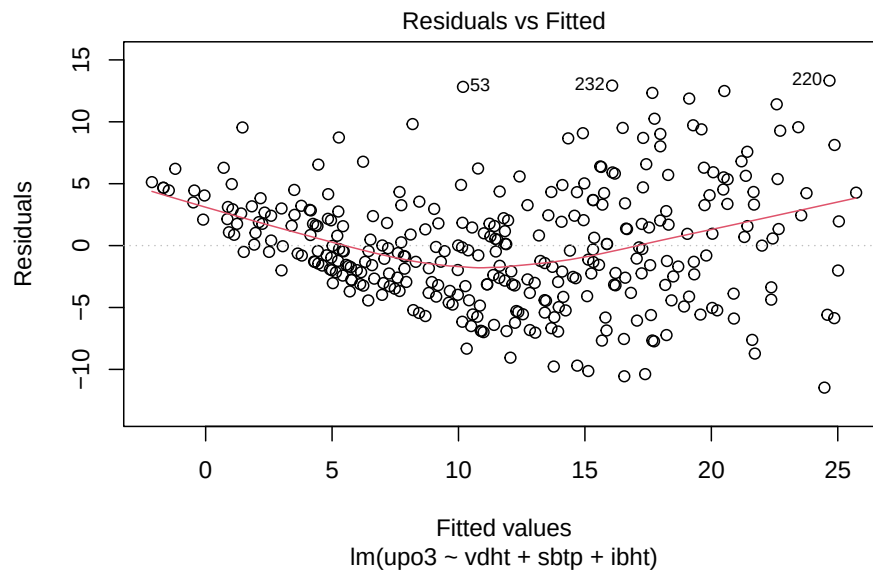
Assume we want to fit a very simple model to these data, we would then start with the following model:

```
lm.ozone <- lm(upo3 ~ vdht + sbtp + ibht, data = ozone)
```

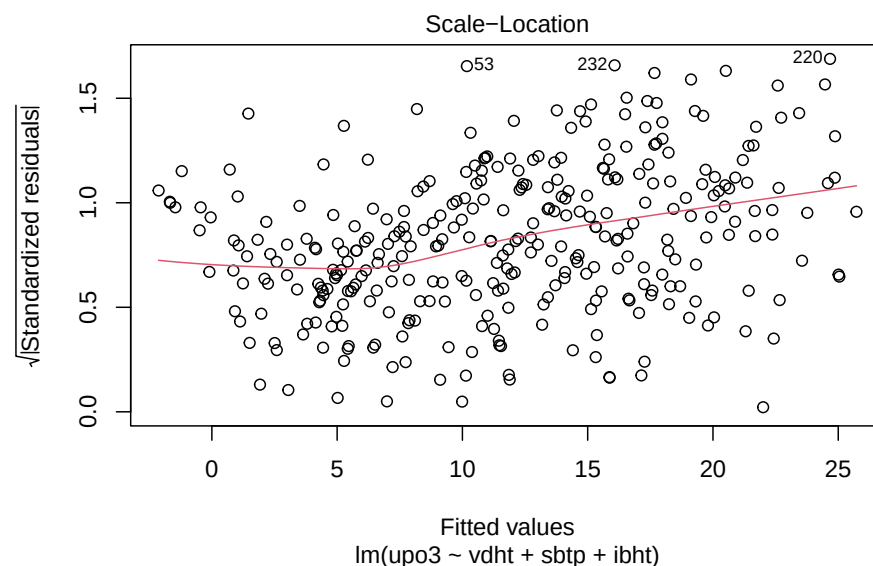
In real settings, we would not fit such a simple model. Indeed, the non-linearities and the heteroscedasticity are very clear. Here we fitted this model for didactic reasons.

Let's now look at the assumptions of this simple model.

```
plot(lm.ozone, which = 1)
```



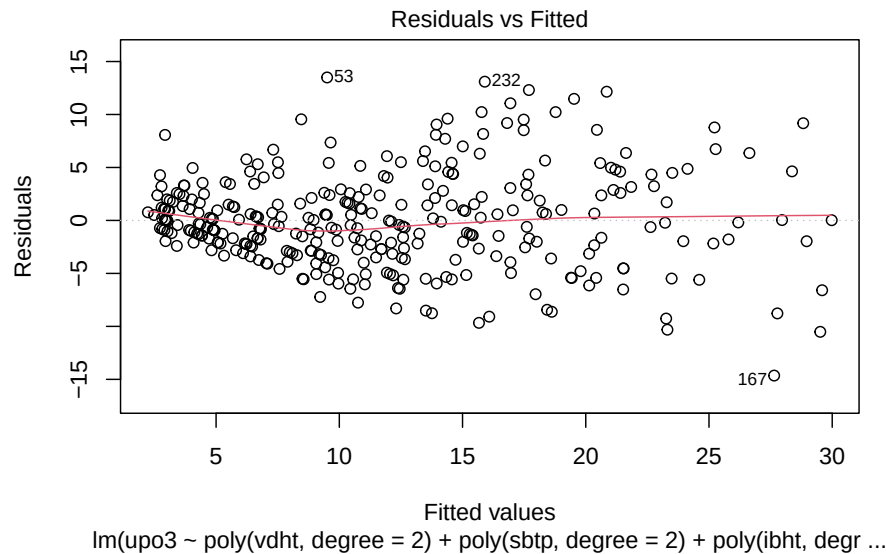
```
plot(lm.ozone, which = 3)
```



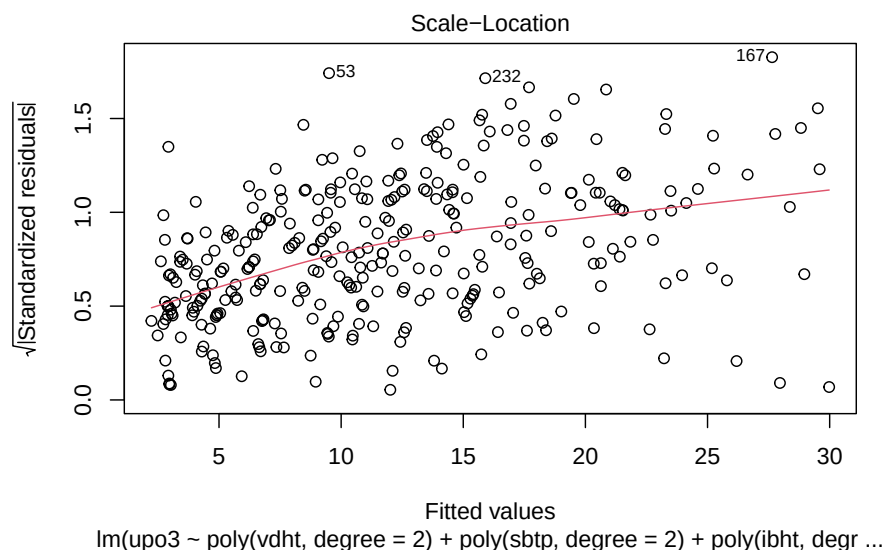
The Tukey-Anscombe plot (first graph) shows a very typical pattern encountered when modelling untransformed “amounts”. Very many observations overlap in the left-hand side of the graph and the smoother is not on zero (but rather has a “U-shape”). In addition, the variance of the residuals increases with the fitted values (second graph).

Let's assume that we are convinced that by simply adding quadratic terms we can get decent residuals plots. Again we do that for didactic reasons only, in reality we would hesitate a second to transform the response variable as it is an amount.

```
lm.ozone.quad <- lm(upo3 ~ poly(vdht, degree = 2) +
  poly(sbtp, degree = 2) +
  poly(ibht, degree = 2), data = ozone)
##
plot(lm.ozone.quad, which = 1)
```



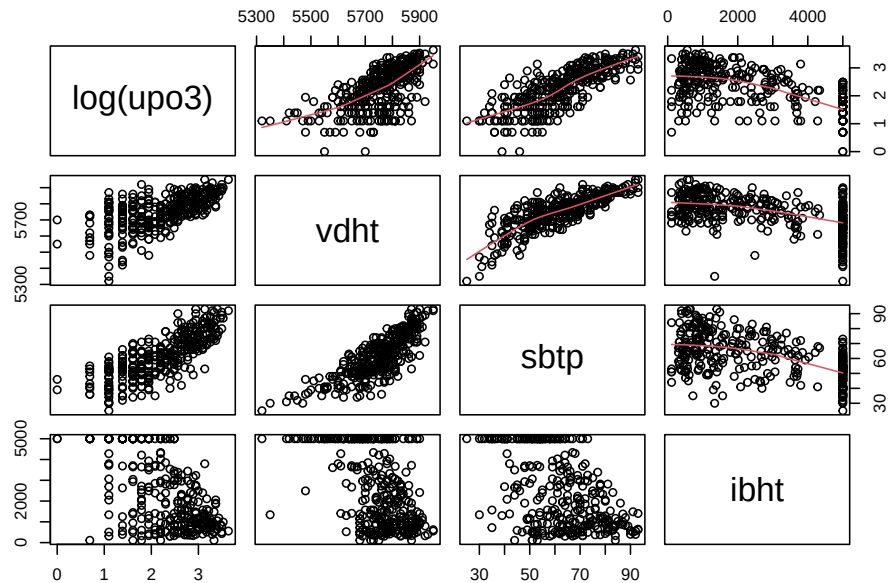
```
plot(lm.ozone.quad, which = 3)
```



Unfortunately, the situation did not improve drastically. Indeed, the typical pattern observed with “amounts” is still present in the Tukey-Anscombe plot (slightly “U-shaped” smoother). Note also that the Scale-Location plot show even more clearly that the variance is not stable, but increasing with the fitted values.

Finally, let's do the proper thing to do: Let's log-transform the response variable to see whether we can then get decent residual plots. Note that as we are applying a transformation, we are also reproducing all the plots of the exploratory graphical analysis.

```
pairs(log(upo3) ~ vdht + sbtp + ibht, data = ozone,
      upper.panel = panel.smooth)
```

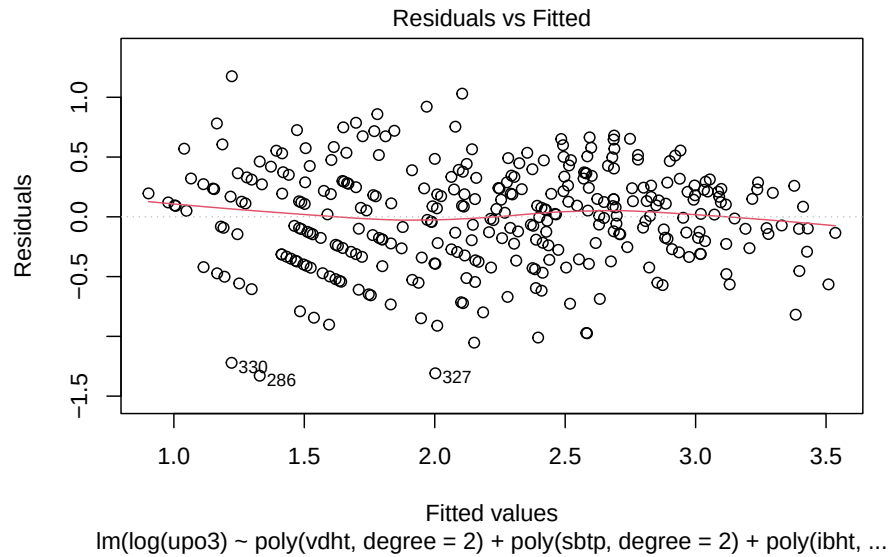


Let's now refit the same model, which contained quadratic effects, with the log-transformed response variable.

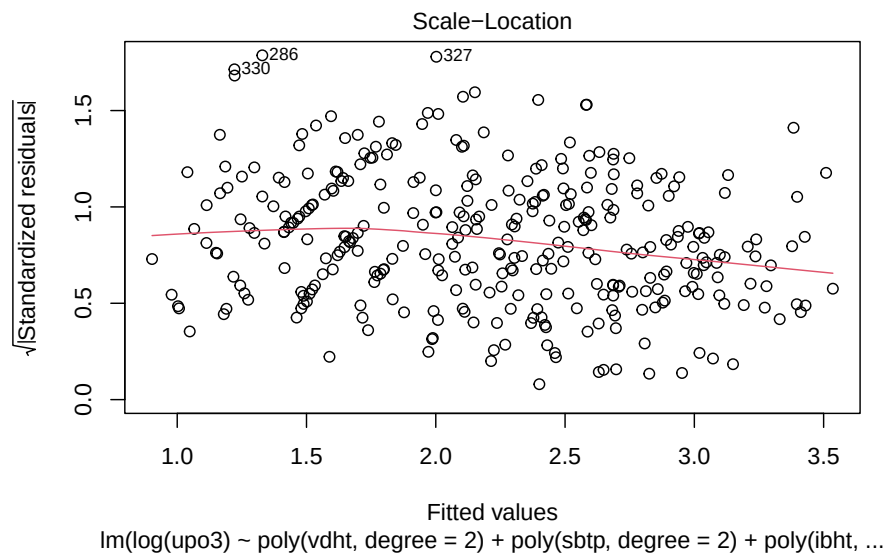
```
lm.ozone.log.quad <- update(lm.ozone.quad, log(upo3) ~ .)
formula(lm.ozone.log.quad)
```

```
log(upo3) ~ poly(vdht, degree = 2) + poly(sbtp, degree = 2) +
  poly(ibht, degree = 2)
```

```
##
##
plot(lm.ozone.log.quad, which = 1)
```



```
plot(lm.ozone.log.quad, which = 3)
```

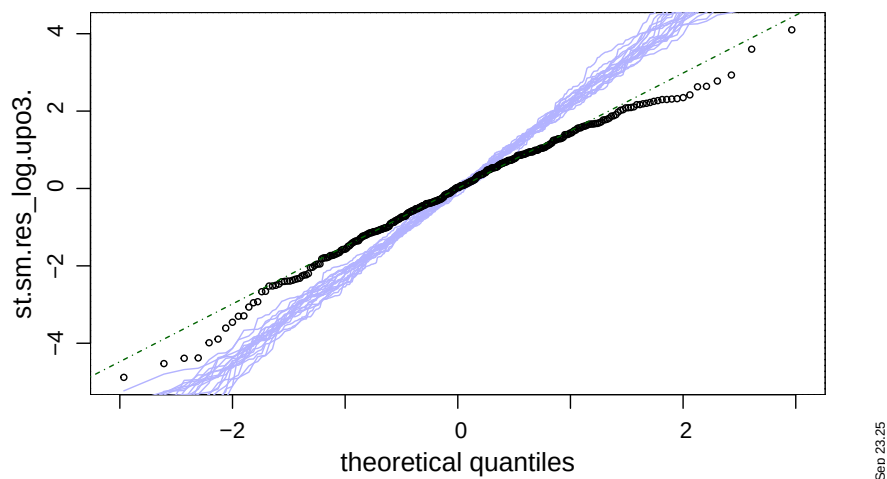


Eventually, the residual plots look much better. The Tukey-Anscombe plot does not show any pattern any more (i.e. the smoother is flat on zero). The variability of the residuals, as seen on the scale-location plot, seems to be fairly constant (maybe slightly decreasing, but ok).

Before looking at the p-values we should still test the normality of the residuals.

```
library(plgraphics)
plregr(lm.ozone.log.quad ,
  plotselect = c(default = 0,
    qq = 2),
  xvar = FALSE)
```

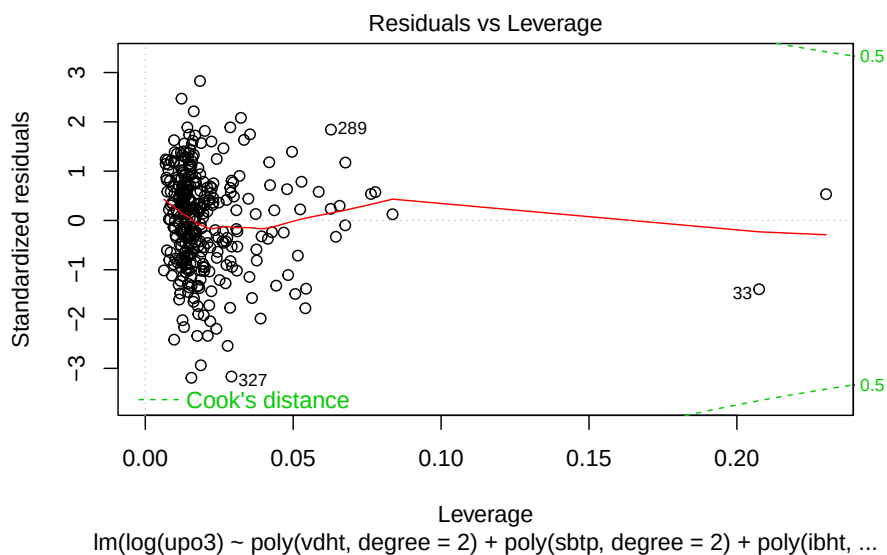
```
log(upo3) ~ poly(vdht, degree = 2) + poly(sbtp, degree = 2) + poly(ibht, degree = 2)
```



There are some observations in the left-hand side that appear to slightly deviate from the normality assumption. We could improve our model (e.g. with cubic terms) and see whether even better residual graphs. In any case, the graphs obtained are to be considered acceptable. In other words, we can look at the p-values and confidence intervals for the *lm.ozone.log.quad* model and trust them.

Finally, we double check that there are no influential observations:

```
plot(lm.ozone.log.quad, which = 5)
```



Excellent, the model assumptions appear to be satisfied and therefore we can look at the p-values.

```
summary(lm.ozone.log.quad)
```

Call:

```
lm(formula = log(upo3) ~ poly(vdht, degree = 2) + poly(sbtp,
    degree = 2) + poly(ibht, degree = 2), data = ozone)

Residuals:
    Min       1Q   Median       3Q      Max
-1.3291 -0.2717  0.0511  0.2759  1.1758

Coefficients:
                Estimate Std. Error t value Pr(>|t|)
(Intercept)         2.2130     0.0231   95.79 < 2e-16 ***
poly(vdht, degree = 2)1 -1.6378     0.7475   -2.19  0.02916 *
poly(vdht, degree = 2)2 -0.3699     0.5900   -0.63  0.53108
poly(sbtp, degree = 2)1  9.4548     0.7765   12.18 < 2e-16 ***
poly(sbtp, degree = 2)2  0.7014     0.5805    1.21  0.22778
poly(ibht, degree = 2)1 -4.3351     0.5039   -8.60  3.4e-16 ***
poly(ibht, degree = 2)2 -1.5152     0.4320   -3.51  0.00052 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.42 on 323 degrees of freedom
Multiple R-squared:  0.691, Adjusted R-squared:  0.686
F-statistic: 121 on 6 and 323 DF, p-value: <2e-16
```

3 Intrepration of coefficients in models with transformation

How can we interpret the regression coefficients of a model where the response variable and or the predictors were log-transformed?

3.1 log(y) case

Let's take the example of the ozone concentration. Let's assume we fit the following model where the response variable has been log-transformed.

```
lm.ozone.simple <- lm(log(upo3) ~ sbtp, data = ozone)
summary(lm.ozone.simple)
```

```
Call:
lm(formula = log(upo3) ~ sbtp, data = ozone)

Residuals:
    Min       1Q   Median       3Q      Max
-1.5763 -0.2708  0.0531  0.3021  1.2763

Coefficients:
                Estimate Std. Error t value Pr(>|t|)
(Intercept)  -0.28247     0.11325   -2.49   0.013 *
sbtp           0.04041     0.00179   22.63 <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Residual standard error: 0.47 on 328 degrees of freedom
Multiple R-squared: 0.61, Adjusted R-squared: 0.608
F-statistic: 512 on 1 and 328 DF, p-value: <2e-16

In this model, we see that by increasing *sbtp* (i.e. air temperature) by one unit, we obtain an increase of 0.04 units of the log-transformed response variable. These results are not really meaningful to us.

Fortunately, there is a simple “trick” to use in this situations: we simply take the inverse transformation and interpret the coefficient in a multiplicative manner.

```
exp(coef(lm.ozone.simple)["sbtp"]) %>%  
  print(digits = 3)
```

```
sbtp  
1.04
```

This implies that when we increase the variable *sbtp* by one unit, the response variable is **multiplied** by 1.04. In other words, the response variable increases by 4%.

4 Appendix

4.1 $\log(y)$ are “multiplicative models”

Why can models where the response variable is log-transformed be considered *multiplicative models*?

As a matter of fact, up to this point, we said that linear models are additive in their coefficients. Indeed, the “simple ozone model” can be formally represented by the following equation:

$$\ln(\text{ozone.conc}) = \beta_1 + \beta_2 \cdot \text{air_temp} + \varepsilon$$

Let’s exponentiate both side of this equation (i.e. take the inverse function of the natural logarithm).

$$\text{ozone.conc} = \exp(\beta_1 + \beta_2 \cdot \text{air_temp} + \varepsilon)$$

We reformulate this equation slightly:

$$\text{ozone.conc} = \exp(\beta_1) \cdot \exp(\beta_2 \cdot \text{air_temp}) \cdot \exp(\varepsilon)$$

This means that by increasing air temperature by one unit, the response variable is **multiplied** by $\exp(\beta_2)$ in the original scale (i.e. $\exp(0.04) = 1.04$). In other words, there is an increase in 4% in the response variable in the original scale.

This way to interpret the coefficient is very handy as in most situations we don’t really need to exponentiate the regression coefficient. Indeed, in most situations $\exp(\beta) \approx 1 + \beta$ ¹.

So, here we can directly read the effect of air temperature from the summary output itself.

4.2 “Nature” is very often multiplicative (especially for amounts)

Let’s assume you are modelling the price of some products in a food store. You are interested in the difference in price between “low-priced” products and “normal” products. A “low-priced” yoghurt may cost 0.30 CHF less than the “normal” yoghurt. On the other hand, a “low-priced steak may cost 5.00 CHF less than the “normal” one. The important point here is that despite the absolute difference in price being very different among products **there is always a -10% difference between low-priced and normal products**. This is why we model “log(price)”.

You can take other examples where you want to model an amount and you will recognise that in most cases a multiplicative model is appropriate.

4.3 $\log(y) \sim \log(x)$ case

You may remember that for the “farms” model we decided to take the log of both, the response variable and the predictor. How should we proceed to interpret the regression coefficient?

```
lm.farms <- lm(log(revenue) ~ log(expenditure) + region + industry,
               data = d.farms)
summary(lm.farms)
```

¹The approximation $\exp(\beta) \approx 1 + \beta$ works well when β is close to zero, which is very often the case.

```
Call:
lm(formula = log(revenue) ~ log(expenditure) + region + industry,
    data = d.farms)
```

```
Residuals:
    Min       1Q   Median       3Q      Max
-1.4762 -0.1739  0.0317  0.2206  1.4476
```

```
Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)    0.89120    0.47013     1.90  0.0587 .
log(expenditure) 0.92774    0.01828    50.76 <2e-16 ***
region        -0.00178    0.00354     -0.50  0.6152
industrymixed -0.12543    0.05410     -2.32  0.0209 *
industrysh&cat -0.12206    0.05599     -2.18  0.0298 *
industrysheep -0.17628    0.05636     -3.13  0.0019 **
industrywheat  0.02197    0.07485      0.29  0.7692
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 0.36 on 444 degrees of freedom
Multiple R-squared:  0.868, Adjusted R-squared:  0.866
F-statistic: 488 on 6 and 444 DF, p-value: <2e-16
```

In this case, the transformation is not needed. As we can directly interpret the regression coefficient as percentage changes for both variables.

```
coef(lm.farms)["log(expenditure)"]
```

```
log(expenditure)
0.93
```

If you increase *expenditure* by 1% percent, your *revenue* will increase by 0.93%.

4.4 Other advantages of log(y) models

Another very practical advantage of models where the response variable is log-transformed is that the fitted and predicted values in the original space will always be equal or greater than zero.

Indeed, you really want predicted amounts to be strictly positive numbers. Having a negative weight, height or salary is most cases nonsensical.

Obviously, to get predicted or fitted values in the original (untransformed space) we simply need to apply the inverse transformation.

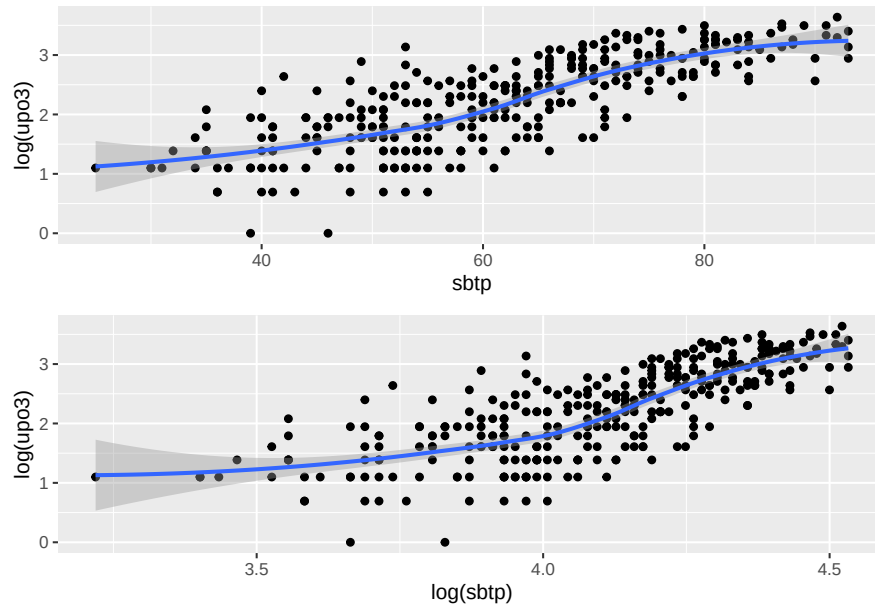
```
exp(fitted(lm.ozone.log.quad)[1:5])
```

```
1 2 3 4 5
5.2 6.5 8.3 4.8 5.6
```

4.5 Why not taking log of e.g. sbtp? (*)

In the previous model we modelled the predictor *sbtp*, which is the “air temperature”, without transformation. In theory, “air temperature” is also an amount and it should be transformed. So, why was it not transformed.

In some cases, a transformation does not make any real changes and the relationship remains not linear. Let's display this situation with *sbtp*.



These two graphs clearly show that transforming *sbtp* does not really change the situation. In these cases we may prefer to leave the predictor untransformed, which may facilitate the interpretation of the coefficients.

5 Session Information

```
sessionInfo()
```

```
R version 4.5.1 (2025-06-13)
```

```
Platform: aarch64-apple-darwin20
```

```
Running under: macOS Sequoia 15.6.1
```

```
Matrix products: default
```

```
BLAS: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
```

```
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK vers
```

```
locale:
```

```
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
```

```
time zone: Europe/Zurich
```

```
tzcode source: internal
```

```
attached base packages:
```

```
[1] stats      graphics  grDevices  utils      datasets  methods    base
```

```
other attached packages:
```

```
[1] gridExtra_2.3 plgraphics_1.3 gss_2.2-9      dplyr_1.1.4    ggplot2_3.5.2
```

```
[6] knitr_1.50
```

```
loaded via a namespace (and not attached):
```

```
[1] Matrix_1.7-3      gtable_0.3.6      compiler_4.5.1    Rcpp_1.1.0
[5] tidyselect_1.2.1  tinytex_0.57      splines_4.5.1     scales_1.4.0
[9] boot_1.3-31       yaml_2.3.10       fastmap_1.2.0     lattice_0.22-7
[13] R6_2.6.1          labeling_0.4.3    generics_0.1.4    rbibutils_2.3
[17] MASS_7.3-65       tibble_3.3.0      nloptr_2.2.1      minqa_1.2.8
[21] pillar_1.11.0     RColorBrewer_1.1-3 rlang_1.1.6       xfun_0.52
[25] cli_3.6.5         withr_3.0.2       magrittr_2.0.3    mgcv_1.9-3
[29] Rdpack_2.6.4      digest_0.6.37     grid_4.5.1        rstudioapi_0.17.1
[33] lme4_1.1-37       lifecycle_1.0.4   nlme_3.1-168      reformulas_0.4.1
[37] vctr_0.6.5        evaluate_1.0.4    glue_1.8.0        farver_2.1.2
[41] survival_3.8-3    rmarkdown_2.29    chron_2.3-62      tools_4.5.1
[45] pkgconfig_2.0.3   htmltools_0.5.8.1
```